



哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

規格嚴格



功夫到家

1920 — 2017

软件构造实践

实验二

2026春

哈尔滨工业大学（深圳）



实验目的

- 掌握跨平台功能迁移的基础思路
- 掌握Android Studio基础操作

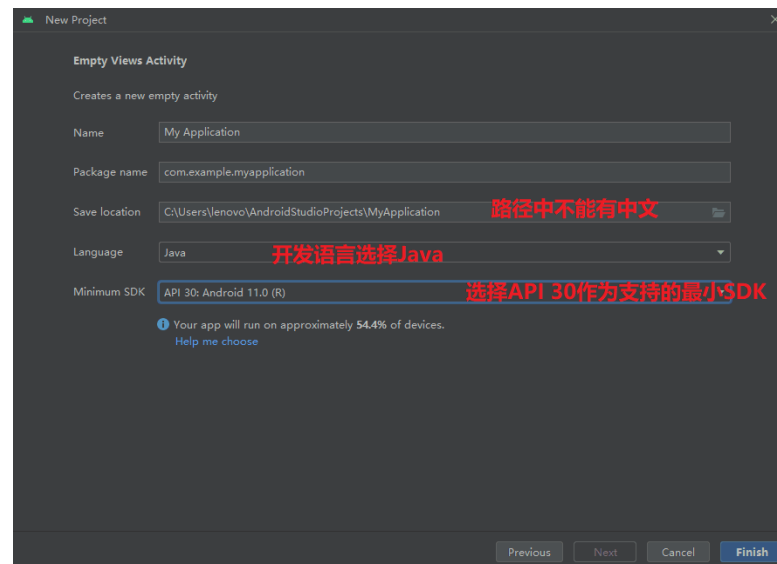
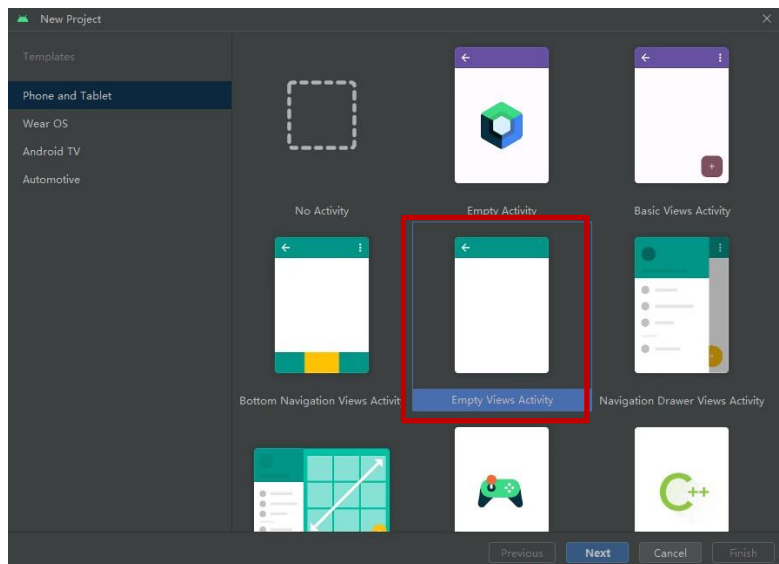


实验内容

1. 创建Android项目
2. 迁移Windows代码
3. 在AS模拟器中运行并展示游戏界面

创建Android项目

创建 “Empty Views Activity” 项目





创建Android项目

➤ 新建Empty Views Activity

New Project

Empty Views Activity

Creates a new empty activity

Name: AircraftWar

Package name: edu.hitsz

Save location: D:\AircraftWar

Language: Java

Minimum SDK: API 30 ("R"; Android 11.0)

ⓘ Your app will run on approximately 83.0% of devices.
[Help me choose](#)

Build configuration language: Groovy DSL (build.gradle)

Previous Next Cancel Finish

➤ 复制Windows代码到Android项目，资源文件（图片）复制到res/drawable文件夹下



Android项目结构

以“Android”模式展示项目结构

- **app/manifests目录**

AndroidManifest.xml文件：提供软件包名，描述应用组件，权限声明等

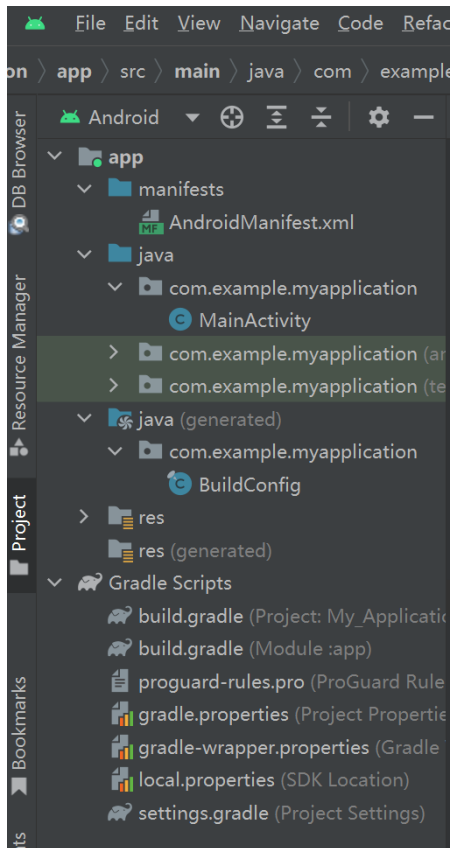
- **app/java目录**

该目录用于存放Android应用项目中的所有Java代码文件。Java代码以用户声明的包进行自动地组织。其中MainActivity.java是默认的首个被运行的代码文件。

- **app/res目录：**

该目录存放整个项目经常使用的资源文件，称为资源目录。该目录包括项目中使用到的所有图标、图片、布局、声音、字符串、颜色、样式等资源参数描述文件。

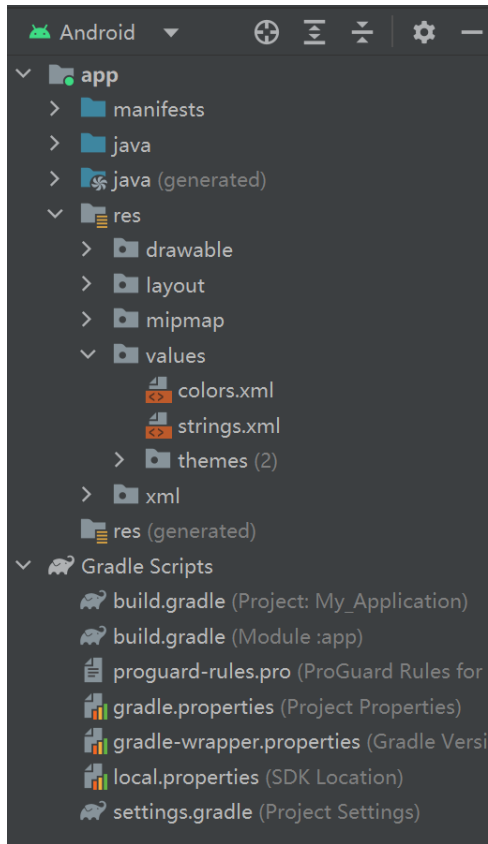
需要注意的是，res目录中的所有文件名只能是以 a~z、0~9或 “_” 字符命名，不能包含大写字母，且必须以字母开头，否则会导致错误。





Android项目结构

res目录结构



- **图片资源目录：**以drawable开头的目录用来存放图片文件，有时还可以存放一些其他的drawable类型的XML文件。
- **图标资源目录：**以mipmap开头的目录用来存放应用的图标。应用项目会根据不同分辨率设备，启动相应分辨率的mipmap目录下的图标。
- **layout目录：**该目录存放应用项目的布局文件，文件类型为XML格式。新建项目时系统会自动创建一个activity_main.xml文件。
- **values目录：**该目录存放所有XML格式的资源描述文件，一般根据英文含义来命名。例如：
 - colors.xml：定义颜色资源。
 - string.xml：定义字符串资源。
 - styles.xml：定义主题资源。



Android项目结构

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.MyApplication"
        tools:targetApi="31">

        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

    </application>
</manifest>
```

- 每个 Android 项目必须有一个 AndroidManifest.xml 文件，当创建一个新的应用项目时，系统会自动生成此文件。作用是：
 - 描述应用项目的每个组件的类名称和组件能力（属性），帮助Android 系统了解这些组件以及在何种条件下可以启动这些组件。
 - 声明应用项目自身应该具有的权限，如应用程序需要访问系统功能（短信、联系人、相机和网络等），需要获取相关权限。



Android项目结构

AndroidManifest.xml文件说明

代码元素	说 明
manifest	xml文件的根结点，包含了package中所有的内容
xmlns:android	命名空间的声明。使得Android中各种标准属性能在文件中使用。
uses-sdk	声明应用程序所使用的Android SDK版本
application	application 级别组件的根结点。声明一些全局或默认的属性，如标签、图标、必要的权限等。
android:icon	应用程序图标
android:label	应用程序名称
activity	Activity 是一个应用程序与用户交互的图形界面。每一个 Activity 必须有一个< activity >标记对应。
android:name	应用程序默认启动的活动程序 Activity 界面
intent-filter	声明一组组件支持的Intent值。在Android中，组件之间可以相互调用，协调工作，Intent提供组件之间通讯所需要的相关信息
action	声明目标组件执行的Intent动作。
category	指定目标组件支持的Intent 类别



Android项目结构

Gradle Scripts

Android 应用程序采用 Gradle 作为构建工具

build.gradle(项目级别): 是项目全局的gradle构建脚本, 该文件的内容通常不需要修改。

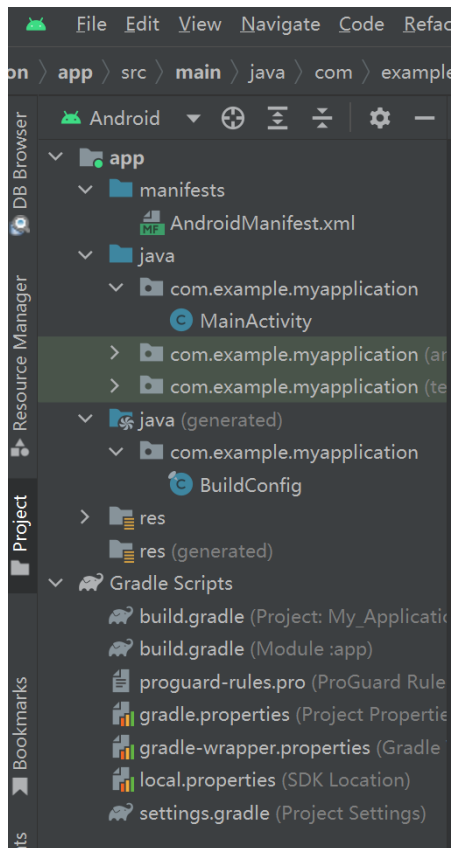
build.gradle (模块级别): app模块的gradle构建脚本, 一般用来管理app包名以及添加和修改依赖库。

gradle.properties: 全局的gradle配置文件, 一般不需要修改。

gradle-wrapper.properties: 配置使用的gradle版本、保存地址以及下载地址等。

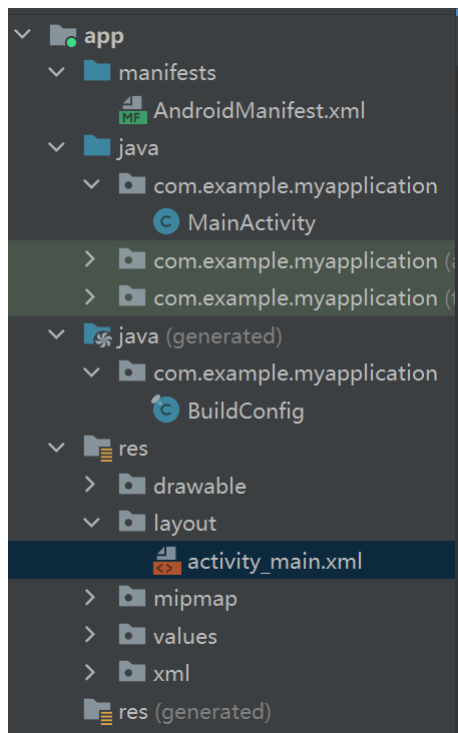
local.properties: 该文件用于指定本机中的Android SDK路径, 由系统自动生成。当Android SDK位置变化时, 需要在该文件更新路径。

settings.gradle: 该文件用于指定项目中所有引入的模块。



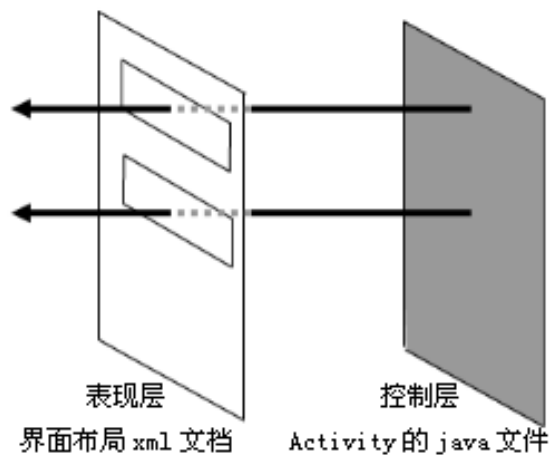


Android项目结构



Android应用程序结构分析

- 一个简单的Android应用程序通常由Activity类程序（Java程序）和用户界面布局XML文件组成。
- 在Android应用程序中，逻辑控制层与表现层是分开的设计的。逻辑控制层由Java应用程序实现，表现层由XML文档描述。





MainActivity.java

```

1 package com.example.HelloAndroid;      ← 包声明语句

2 import androidx.appcompat.AppCompatActivity; ← 导入包
3 import android.os.Bundle;

4 public class MainAndroid extends AppCompatActivity ← 类标志
5 {                                             ← 类名
6     public void onCreate(Bundle savedInstanceState) ← 重写 onCreate()方法
7     {
8         super.onCreate(savedInstanceState); ← 调用父类 Activity 的 onCreate()方法
9         setContentView(R.layout.activity_main); ← 在屏幕上显示内容的方法
10    }
11 }

```



任务

- 安装Android Studio Panda，配置开发环境；
- 在Android Studio中创建一个名为“AircarftWar”的项目，注意项目路径**不要有中文**；
- 项目语言为Java；
- 支持的最小API级别为30；
- 使用模拟器运行此应用程序；
- 熟悉Android Studio开发环境和应用程序的目录结构；

说明：以上内容仅做参考，sdk版本、开发语言、开发环境均可自行选择；



迁移Windows代码

➤ 移除Java SE相关代码，使用Android技术重写

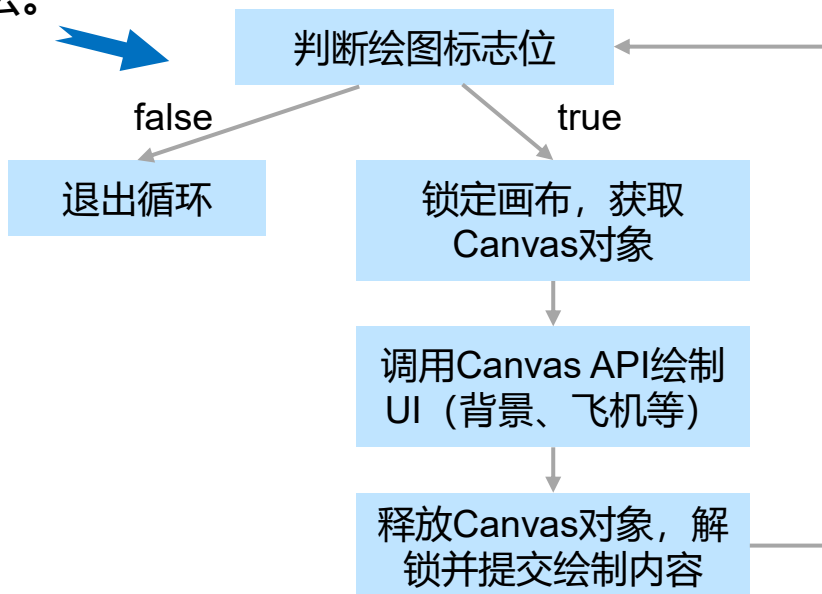
功能场景	Windows实现方式	Android实现方式
输入交互	HeroController类 (鼠标输入)	在BaseGame类中新增触屏事件处理逻辑 setOnTouchListener(<code>new View.OnTouchListener()</code> { <code>@Override</code> <code>public boolean onTouch</code> (View view, MotionEvent motionEvent) { } });
图片加载	ImageManager.java ImageIO.read() BufferedImage	BitmapFactory + 资源ID (图片放 res/drawable) Bitmap playerBmp = BitmapFactory. <code>decodeResource</code> (<code>context.getResources()</code> , <code>R.drawable.player</code>);
入口+界面	Main.java JFrame/JPanel等 Swing组件	Activity + SurfaceView 项目入口为MainActivity.java BaseGame重构为SurfaceView 排行榜页面和难度选择页面（待实现）
音频处理	AudioSystem	MediaPlayer、SoundPool (待实现)



迁移Windows代码

➤ 将BaseGame重构为SurfaceView的子类

- ❑ 自定义SurfaceView类，实现SurfaceHolder.Callback和Runnable接口，初始化核心成员(SurfaceHolder、绘制线程、绘制标志位)；
- ❑ 重写SurfaceCreated方法，加载图片，设置绘制标志位为true，启动游戏绘制线程；
- ❑ 重写SurfaceChanged方法，获取屏幕宽高；
- ❑ 重写SurfaceDestroyed方法，设置绘制标志位为false，终止游戏绘制；
- ❑ 实现 Runnable 的 run () 方法。





SurfaceView实例

- 如何使用SurfaceView

1. 创建SurfaceView

创建自定义的SurfaceView继承自SurfaceView,并实现两个接口:

SurfaceHolder.Callback和Runnable

```
public class MySurfaceView extends SurfaceView implements  
    SurfaceHolder.Callback, Runnable
```

2. 初始化SurfaceView

在自定义的SurfaceView的构造函数中, 定义并初始化如下的成员变量

```
public MySurfaceView(Context context, AttributeSet attrs, int defStyle) {  
    super(context, attrs, defStyle);  
    initView();  
}  
  
private void initView() {  
    mHolder = getHolder();  
    mHolder.addCallback(this);  
    setFocusable(true);  
    setFocusableInTouchMode(true);  
    this.setKeepScreenOn(true);  
    //mHolder.setFormat(PixelFormat.OPAQUE);  
}
```



SurfaceView实例

- 如何使用SurfaceView

3. 实现SurfaceHolder.Callback接口

```
@Override
public void surfaceCreated(SurfaceHolder holder) {
    mIsDrawing = true;
    new Thread(this).start();
}

@Override
public void surfaceChanged(SurfaceHolder holder,
                           int format, int width, int height) {
}

@Override
public void surfaceDestroyed(SurfaceHolder holder) {
    mIsDrawing = false;
}
```

- 在surfaceCreated()方法中开启子线程进行绘制，子线程使用while(mIsDrawing)的循环来不停的进行绘制；
- 在surfaceDestroyed()方法中设置mIsDrawing为false结束绘制；



SurfaceView实例

- 如何使用SurfaceView

4. 实现Runnable接口

```
@Override
public void run() {
    while (mIsDrawing) {
        draw();
    }
}
// 绘图操作
private void draw() {
    try {
        mCanvas = mHolder.lockCanvas();
        // draw sth 绘制过程
    } catch (Exception e) {
    } finally {
        if (mCanvas != null)
            mHolder.unlockCanvasAndPost(mCanvas); // 保证每次都提交绘图内容
    }
}
```

- 在绘制的逻辑中通过lockCanvas()方法获取Canvas对象进行绘制，通过unlockCanvasAndPost(mCanvas)方法对画布内容进行提交。



在MainActivity中加载BaseGame

```
public class MainActivity extends AppCompatActivity {  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
  
        //setContentView(R.layout.activity_main);  
        //默认使用简单模式  
        setContentView(new EasyGame(this));  
        .....  
    }  
}
```



**同学们
请开始实验吧！**